SQL Solution Manual (Part 1)

Levels 1 through 4 (Questions 1–110) | Verified with Expected Output Data

L1 — SQL Fundamentals (Questions 1–25)

> **Focus:** SELECT, WHERE, operators, DISTINCT, ORDER BY, LIMIT, basic functions.

Q1. Retrieve all columns from the employees table.

```
SELECT * FROM employees;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
1	Sarah Connor	sarah@company.com	120000	10	NULL	2020-01-15	Director
2	Alice Smith	alice@company.com	60000	10	1	2023-02-10	Developer
3	Brian	brian@company.com	50000	20	1	2024-05-20	HR Specialist

Q2. Retrieve only employee names and salaries.

```
SELECT emp_name, salary FROM employees;
```

SAMPLE OUTPUT:

emp_name	salary
Sarah Connor	120000
Alice Smith	60000
Brian	50000

Q3. Retrieve all unique department IDs.

```
SELECT DISTINCT dept_id FROM employees WHERE dept_id IS NOT NULL;
```

SAMPLE OUTPUT:

dept_id
10
20
30

Q4. Find employees whose salary is greater than 50,000.

```
SELECT * FROM employees WHERE salary > 50000;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
1	Sarah Connor	sarah@company.com	120000	10	NULL	2020-01-15	Director
2	Alice Smith	alice@company.com	60000	10	1	2023-02-10	Developer
6	Diana	diana@company.com	75000	30	1	2026-08-05	Senior Analyst

Q5. Find employees whose salary is exactly 60,000.

```
SELECT * FROM employees WHERE salary = 60000;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
2	Alice Smith	alice@company.com	60000	10	1	2023-02-10	Developer
7	Evan	evan@company.com	60000	20	3	2023-12-01	Recruiter

Q6. Find employees whose salary is less than 40,000.

```
SELECT * FROM employees WHERE salary < 40000;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
5	Dan	dan@company.com	35000	30	1	2022-11-01	Analyst

Q7. Find employees who belong to department 10.

```
SELECT * FROM employees WHERE dept_id = 10;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
1	Sarah Connor	sarah@company.com	120000	10	NULL	2020-01-15	Director
2	Alice Smith	alice@company.com	60000	10	1	2023-02-10	Developer
4	Alan	alan@company.com	40000	10	2	2024-01-15	Junior Dev

Q8. Find employees who do not belong to department 10.

```
SELECT * FROM employees WHERE dept_id != 10;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
3	Brian	brian@company.com	50000	20	1	2024-05-20	HR Specialist
5	Dan	dan@company.com	35000	30	1	2022-11-01	Analyst
6	Diana	diana@company.com	75000	30	1	2026-08-05	Senior Analyst

Q9. Find employees whose salary is between 40,000 and 70,000.

```
SELECT * FROM employees WHERE salary BETWEEN 40000 AND 70000;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
2	Alice Smith	alice@company.com	60000	10	1	2023-02-10	Developer
3	Brian	brian@company.com	50000	20	1	2024-05-20	HR Specialist
4	Alan	alan@company.com	40000	10	2	2024-01-15	Junior Dev

Q10. Find employees whose department is either 10, 20, or 30.

```
SELECT * FROM employees WHERE dept_id IN (10, 20, 30);
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
1	Sarah Connor	sarah@company.com	120000	10	NULL	2020-01-15	Director
2	Alice Smith	alice@company.com	60000	10	1	2023-02-10	Developer
3	Brian	brian@company.com	50000	20	1	2024-05-20	HR Specialist

Q11. Find employees whose name starts with 'A'.

```
SELECT * FROM employees WHERE emp_name LIKE 'A%';
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
2	Alice Smith	alice@company.com	60000	10	1	2023-02-10	Developer
4	Alan	alan@company.com	40000	10	2	2024-01-15	Junior Dev

Q12. Find employees whose name ends with 'n'.

```
SELECT * FROM employees WHERE emp_name LIKE '%n';
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
3	Brian	brian@company.com	50000	20	1	2024-05-20	HR Specialist
4	Alan	alan@company.com	40000	10	2	2024-01-15	Junior Dev
5	Dan	dan@company.com	35000	30	1	2022-11-01	Analyst

Q13. Find employees whose name contains 'an'.

```
SELECT * FROM employees WHERE emp_name LIKE '%an%';
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
3	Brian	brian@company.com	50000	20	1	2024-05-20	HR Specialist
4	Alan	alan@company.com	40000	10	2	2024-01-15	Junior Dev
5	Dan	dan@company.com	35000	30	1	2022-11-01	Analyst

Q14. Find employees whose manager ID is NULL.

```
SELECT * FROM employees WHERE manager_id IS NULL;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
1	Sarah Connor	sarah@company.com	120000	10	NULL	2020-01-15	Director

Q15. Find employees who have a manager.

```
SELECT * FROM employees WHERE manager_id IS NOT NULL;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
2	Alice Smith	alice@company.com	60000	10	1	2023-02-10	Developer
3	Brian	brian@company.com	50000	20	1	2024-05-20	HR Specialist
4	Alan	alan@company.com	40000	10	2	2024-01-15	Junior Dev

Q16. Sort employees by salary in ascending order.

```
SELECT * FROM employees ORDER BY salary ASC;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
5	Dan	dan@company.com	35000	30	1	2022-11-01	Analyst
4	Alan	alan@company.com	40000	10	2	2024-01-15	Junior Dev
8	Orphan Employee	orphan@company.com	45000	NULL	1	2025-01-01	Consultant

Q17. Sort employees by salary in descending order.

```
SELECT * FROM employees ORDER BY salary DESC;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
10	Ian	ian@company.com	150000	40	1	2021-06-15	VP Sales
1	Sarah Connor	sarah@company.com	120000	10	NULL	2020-01-15	Director
6	Diana	diana@company.com	75000	30	1	2026-08-05	Senior Analyst

Q18. Sort employees first by department and then by salary descending.

```
SELECT * FROM employees ORDER BY dept_id ASC, salary DESC;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
8	Orphan Employee	orphan@company.com	45000	NULL	1	2025-01-01	Consultant
1	Sarah Connor	sarah@company.com	120000	10	NULL	2020-01-15	Director
2	Alice Smith	alice@company.com	60000	10	1	2023-02-10	Developer

Q19. Find the five highest-paid employees.

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
10	Ian	ian@company.com	150000	40	1	2021-06-15	VP Sales
1	Sarah Connor	sarah@company.com	120000	10	NULL	2020-01-15	Director
6	Diana	diana@company.com	75000	30	1	2026-08-05	Senior Analyst
2	Alice Smith	alice@company.com	60000	10	1	2023-02-10	Developer
7	Evan	evan@company.com	60000	20	3	2023-12-01	Recruiter

Q20. Find the five lowest-paid employees.

```
SELECT * FROM employees ORDER BY salary ASC LIMIT 5;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
5	Dan	dan@company.com	35000	30	1	2022-11-01	Analyst
4	Alan	alan@company.com	40000	10	2	2024-01-15	Junior Dev
8	Orphan Employee	orphan@company.com	45000	NULL	1	2025-01-01	Consultant
3	Brian	brian@company.com	50000	20	1	2024-05-20	HR Specialist
9	John	john@company.com	50000	10	2	2026-08-10	Developer

Q21. Count the total number of employees.

```
SELECT COUNT(*) FROM employees;
```

SAMPLE OUTPUT:

COUNT(*)
10

Q22. Find the highest salary.

```
SELECT MAX(salary) FROM employees;
```

SAMPLE OUTPUT:

MAX(salary)
150000

Q23. Find the lowest salary.

```
SELECT MIN(salary) FROM employees;
```

SAMPLE OUTPUT:

MIN(salary)
35000

Q24. Find the average salary.

```
SELECT AVG(salary) FROM employees;
```

SAMPLE OUTPUT:

AVG(salary)
68500.0

Q25. Find the total salary paid to all employees.

```
SELECT SUM(salary) FROM employees;
```

SAMPLE OUTPUT:

SUM(salary)
685000

L2 — Basic SQL + Functions (Questions 26–50)

> **Focus:** Aggregate functions, aliases, expressions, CASE, strings, dates, COALESCE.

Q26. Display employee name and salary with the salary increased by 10%.

```
SELECT emp_name, salary, salary * 1.10 AS increased_salary FROM employees;
```

SAMPLE OUTPUT:

emp_name	salary	increased_salary
Sarah Connor	120000	132000.0
Alice Smith	60000	66000.0
Brian	50000	55000.00000000001

Q27. Display employee name and annual salary.

```
SELECT emp_name, salary * 12 AS annual_salary FROM employees;
```

SAMPLE OUTPUT:

emp_name	annual_salary
Sarah Connor	1440000
Alice Smith	720000
Brian	600000

Q28. Display employee name and a calculated column showing whether the employee earns more than 60,000.

```
SELECT emp_name, CASE WHEN salary > 60000 THEN 'Yes' ELSE 'No' END AS
earns_more_than_60k FROM employees;
```

SAMPLE OUTPUT:

emp_name	earns_more_than_60k
Sarah Connor	Yes
Alice Smith	No
Brian	No

Q29. Find the number of employees who earn more than 50,000.

```
SELECT COUNT(*) FROM employees WHERE salary > 50000;
```

SAMPLE OUTPUT:

COUNT(*)
5

Q30. Find the number of employees who belong to department 20.

```
SELECT COUNT(*) FROM employees WHERE dept_id = 20;
```

SAMPLE OUTPUT:

COUNT(*)
2

Q31. Find the average salary of employees in department 10.

```
SELECT AVG(salary) FROM employees WHERE dept_id = 10;
```

SAMPLE OUTPUT:

AVG(salary)
67500.0

Q32. Find the total salary paid to employees in department 30.

```
SELECT SUM(salary) FROM employees WHERE dept_id = 30;
```

SAMPLE OUTPUT:

SUM(salary)
110000

Q33. Find the difference between the highest and lowest salary.

```
SELECT MAX(salary) - MIN(salary) AS salary_diff FROM employees;
```

SAMPLE OUTPUT:

salary_diff
115000

Q34. Count the number of employees whose manager ID is not NULL.

```
SELECT COUNT(manager_id) FROM employees;
```

SAMPLE OUTPUT:

COUNT(manager_id)
9

Q35. Count the number of distinct departments.

```
SELECT COUNT(DISTINCT dept_id) FROM employees;
```

SAMPLE OUTPUT:

COUNT(DISTINCT dept_id)
4

Q36. Display employee names in uppercase.

```
SELECT UPPER(emp_name) AS emp_name_upper FROM employees;
```

SAMPLE OUTPUT:

emp_name_upper
SARAH CONNOR
ALICE SMITH
BRIAN

Q37. Display employee names in lowercase.

```
SELECT LOWER(emp_name) AS emp_name_lower FROM employees;
```

SAMPLE OUTPUT:

emp_name_lower
sarah connor
alice smith
brian

Q38. Find the length of each employee's name.

```
SELECT emp_name, LENGTH(emp_name) AS name_length FROM employees;
```

SAMPLE OUTPUT:

emp_name	name_length
Sarah Connor	12
Alice Smith	11
Brian	5

Q39. Find employees whose names contain exactly five characters.

```
SELECT * FROM employees WHERE LENGTH(emp_name) = 5;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
3	Brian	brian@company.com	50000	20	1	2024-05-20	HR Specialist
6	Diana	diana@company.com	75000	30	1	2026-08-05	Senior Analyst

Q40. Extract the year from each employee's joining date.

```
SELECT emp_name, YEAR(hire_date) AS join_year FROM employees;
```

SAMPLE OUTPUT:

emp_name	join_year
Sarah Connor	2020
Alice Smith	2023
Brian	2024

Q41. Find employees who joined after January 1, 2023.

```
SELECT * FROM employees WHERE hire_date > '2023-01-01';
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
2	Alice Smith	alice@company.com	60000	10	1	2023-02-10	Developer
3	Brian	brian@company.com	50000	20	1	2024-05-20	HR Specialist
4	Alan	alan@company.com	40000	10	2	2024-01-15	Junior Dev

Q42. Find employees who joined during 2024.

```
SELECT * FROM employees WHERE YEAR(hire_date) = 2024;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
3	Brian	brian@company.com	50000	20	1	2024-05-20	HR Specialist
4	Alan	alan@company.com	40000	10	2	2024-01-15	Junior Dev

Q43. Find the number of employees who joined each year.

```
SELECT YEAR(hire_date) AS join_year, COUNT(*) AS num_employees FROM employees GROUP BY
YEAR(hire_date);
```

SAMPLE OUTPUT:

join_year	num_employees
2020	1
2021	1
2022	1

Q44. Display NULL manager IDs as 'No Manager'.

```
SELECT COALESCE(CAST(manager_id AS CHAR), 'No Manager') AS manager_status FROM
employees;
```

SAMPLE OUTPUT:

manager_status
No Manager
1
1

Q45. Categorize employees as Low, Medium, or High salary using CASE.

```
SELECT emp_name, CASE WHEN salary < 40000 THEN 'Low' WHEN salary BETWEEN 40000 AND
70000 THEN 'Medium' ELSE 'High' END AS salary_category FROM employees;
```

SAMPLE OUTPUT:

emp_name	salary_category
Sarah Connor	High
Alice Smith	Medium
Brian	Medium

Q46. Find the average salary rounded to two decimal places.

```
SELECT ROUND(AVG(salary), 2) FROM employees;
```

SAMPLE OUTPUT:

ROUND(AVG(salary), 2)
68500.0

Q47. Find employees whose salary is an even number.

```
SELECT * FROM employees WHERE salary % 2 = 0;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
1	Sarah Connor	sarah@company.com	120000	10	NULL	2020-01-15	Director
2	Alice Smith	alice@company.com	60000	10	1	2023-02-10	Developer
3	Brian	brian@company.com	50000	20	1	2024-05-20	HR Specialist

Q48. Find employees whose salary is divisible by 5,000.

```
SELECT * FROM employees WHERE salary % 5000 = 0;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
1	Sarah Connor	sarah@company.com	120000	10	NULL	2020-01-15	Director
2	Alice Smith	alice@company.com	60000	10	1	2023-02-10	Developer
3	Brian	brian@company.com	50000	20	1	2024-05-20	HR Specialist

Q49. Find the three most recently joined employees.

```
SELECT * FROM employees ORDER BY hire_date DESC LIMIT 3;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
9	John	john@company.com	50000	10	2	2026-08-10	Developer
6	Diana	diana@company.com	75000	30	1	2026-08-05	Senior Analyst
8	Orphan Employee	orphan@company.com	45000	NULL	1	2025-01-01	Consultant

Q50. Find employees who joined in the current month.

```
SELECT * FROM employees WHERE YEAR(hire_date) = YEAR(CURRENT_DATE) AND MONTH(hire_date)
= MONTH(CURRENT_DATE);
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
6	Diana	diana@company.com	75000	30	1	2026-08-05	Senior Analyst
9	John	john@company.com	50000	10	2	2026-08-10	Developer

L3 — GROUP BY & HAVING (Questions 51–75)

> **Focus:** Aggregation and understanding the difference between WHERE and HAVING.

Q51. Find the number of employees in each department.

```
SELECT dept_id, COUNT(*) AS employee_count FROM employees GROUP BY dept_id;
```

SAMPLE OUTPUT:

dept_id	employee_count
NULL	1
10	4
20	2

Q52. Find the average salary in each department.

```
SELECT dept_id, AVG(salary) AS avg_salary FROM employees GROUP BY dept_id;
```

SAMPLE OUTPUT:

dept_id	avg_salary
NULL	45000.0
10	67500.0
20	55000.0

Q53. Find the maximum salary in each department.

```
SELECT dept_id, MAX(salary) AS max_salary FROM employees GROUP BY dept_id;
```

SAMPLE OUTPUT:

dept_id	max_salary
NULL	45000
10	120000
20	60000

Q54. Find the minimum salary in each department.

```
SELECT dept_id, MIN(salary) AS min_salary FROM employees GROUP BY dept_id;
```

SAMPLE OUTPUT:

dept_id	min_salary
NULL	45000
10	40000
20	50000

Q55. Find the total salary paid by each department.

```
SELECT dept_id, SUM(salary) AS total_salary FROM employees GROUP BY dept_id;
```

SAMPLE OUTPUT:

dept_id	total_salary
NULL	45000
10	270000
20	110000

Q56. Find the number of employees hired in each year.

```
SELECT YEAR(hire_date) AS hire_year, COUNT(*) AS emp_count FROM employees GROUP BY
YEAR(hire_date);
```

SAMPLE OUTPUT:

hire_year	emp_count
2020	1
2021	1
2022	1

Q57. Find the number of employees hired in each month.

```
SELECT MONTH(hire_date) AS hire_month, COUNT(*) AS emp_count FROM employees GROUP BY
MONTH(hire_date);
```

SAMPLE OUTPUT:

hire_month	emp_count
1	3
2	1
5	1

Q58. Find departments having more than five employees.

```
SELECT dept_id FROM employees GROUP BY dept_id HAVING COUNT(*) > 5;
```

SAMPLE OUTPUT:

OUTPUT: (0 ROWS RETURNED)

Q59. Find departments whose average salary exceeds 60,000.

```
SELECT dept_id FROM employees GROUP BY dept_id HAVING AVG(salary) > 60000;
```

SAMPLE OUTPUT:

dept_id
10
40

Q60. Find departments whose total salary exceeds 500,000.

```
SELECT dept_id FROM employees GROUP BY dept_id HAVING SUM(salary) > 500000;
```

SAMPLE OUTPUT:

OUTPUT: (0 ROWS RETURNED)

Q61. Find departments having at least one employee earning more than 100,000.

```
SELECT dept_id FROM employees GROUP BY dept_id HAVING MAX(salary) > 100000;
```

SAMPLE OUTPUT:

dept_id
10
40

Q62. Find departments where the minimum salary is greater than 40,000.

```
SELECT dept_id FROM employees GROUP BY dept_id HAVING MIN(salary) > 40000;
```

SAMPLE OUTPUT:

dept_id
NULL
20
40

Q63. Find departments where the maximum salary is less than 100,000.

```
SELECT dept_id FROM employees GROUP BY dept_id HAVING MAX(salary) < 100000;
```

SAMPLE OUTPUT:

dept_id
NULL
20
30

Q64. Find the number of employees for each combination of department and job role.

```
SELECT dept_id, job_role, COUNT(*) AS emp_count FROM employees GROUP BY dept_id,
job_role;
```

SAMPLE OUTPUT:

dept_id	job_role	emp_count
NULL	Consultant	1
10	Developer	2
10	Director	1

Q65. Find the average salary for each job role.

```
SELECT job_role, AVG(salary) FROM employees GROUP BY job_role;
```

SAMPLE OUTPUT:

job_role	AVG(salary)
Analyst	35000.0
Consultant	45000.0
Developer	55000.0

Q66. Find job roles having more than three employees.

```
SELECT job_role FROM employees GROUP BY job_role HAVING COUNT(*) > 3;
```

SAMPLE OUTPUT:

OUTPUT: (0 ROWS RETURNED)

Q67. Find job roles whose average salary exceeds the company-wide average salary.

```
SELECT job_role FROM employees GROUP BY job_role HAVING AVG(salary) > (SELECT
AVG(salary) FROM employees);
```

SAMPLE OUTPUT:

job_role
Director
Senior Analyst
VP Sales

Q68. Find the department with the highest average salary.

```
SELECT dept_id FROM employees GROUP BY dept_id ORDER BY AVG(salary) DESC LIMIT 1;
```

SAMPLE OUTPUT:

dept_id
40

Q69. Find the department with the lowest average salary.

```
SELECT dept_id FROM employees GROUP BY dept_id ORDER BY AVG(salary) ASC LIMIT 1;
```

SAMPLE OUTPUT:

dept_id
NULL

Q70. Find the department with the highest total salary expenditure.

```
SELECT dept_id FROM employees GROUP BY dept_id ORDER BY SUM(salary) DESC LIMIT 1;
```

SAMPLE OUTPUT:

dept_id
10

Q71. Find the department with the most employees.

```
SELECT dept_id FROM employees GROUP BY dept_id ORDER BY COUNT(*) DESC LIMIT 1;
```

SAMPLE OUTPUT:

dept_id
10

Q72. Find the salary range (MAX - MIN) for every department.

```
SELECT dept_id, MAX(salary) - MIN(salary) AS salary_range FROM employees GROUP BY
dept_id;
```

SAMPLE OUTPUT:

dept_id	salary_range
NULL	0
10	80000
20	10000

Q73. Find departments where the salary range exceeds 50,000.

```
SELECT dept_id FROM employees GROUP BY dept_id HAVING (MAX(salary) - MIN(salary)) >
50000;
```

SAMPLE OUTPUT:

dept_id
10

Q74. Find the year with the highest number of employee hires.

```
SELECT YEAR(hire_date) AS hire_year FROM employees GROUP BY YEAR(hire_date) ORDER BY
COUNT(*) DESC LIMIT 1;
```

SAMPLE OUTPUT:

hire_year
2026

Q75. Find departments where more than 50% of employees earn above 60,000.

```
SELECT dept_id FROM employees GROUP BY dept_id HAVING SUM(CASE WHEN salary > 60000 THEN 1.0 ELSE 0.0 END) / COUNT(*) > 0.5;
```

SAMPLE OUTPUT:

dept_id
40

L4 — JOINS (Questions 76–110)

> **Focus:** INNER JOIN, LEFT JOIN, self joins, multiple joins.

Q76. Display each employee along with their department name.

```
SELECT e.emp_name, d.dept_name FROM employees e INNER JOIN departments d ON e.dept_id = d.dept_id;
```

SAMPLE OUTPUT:

emp_name	dept_name
Sarah Connor	IT
Alice Smith	IT
Brian	HR

Q77. Display each employee along with their manager's name.

```
SELECT e.emp_name AS employee, m.emp_name AS manager FROM employees e LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

SAMPLE OUTPUT:

employee	manager
Sarah Connor	NULL
Alice Smith	Sarah Connor
Brian	Sarah Connor

Q78. Find employees who do not belong to any department.

```
SELECT * FROM employees WHERE dept_id IS NULL;
```

SAMPLE OUTPUT:

emp_id	emp_name	email	salary	dept_id	manager_id	hire_date	job_role
8	Orphan Employee	orphan@company.com	45000	NULL	1	2025-01-01	Consultant

Q79. Find departments that currently have no employees.

```
SELECT d.dept_name FROM departments d LEFT JOIN employees e ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

SAMPLE OUTPUT:

dept_name
Operations
EmptyDept

Q80. Find employees whose department name is 'IT'.

```
SELECT e.emp_name FROM employees e JOIN departments d ON e.dept_id = d.dept_id WHERE
d.dept_name = 'IT';
```

SAMPLE OUTPUT:

emp_name
Sarah Connor
Alice Smith
Alan

Q81. Find employees working in departments located in 'Hyderabad'.

```
SELECT e.emp_name FROM employees e JOIN departments d ON e.dept_id = d.dept_id WHERE
d.location = 'Hyderabad';
```

SAMPLE OUTPUT:

emp_name
Sarah Connor
Alice Smith
Alan

Q82. Find the number of employees in each department using a join.

```
SELECT d.dept_name, COUNT(e.emp_id) AS emp_count FROM departments d LEFT JOIN employees e ON d.dept_id = e.dept_id GROUP BY d.dept_name;
```

SAMPLE OUTPUT:

dept_name	emp_count
EmptyDept	0
Finance	2
HR	2

Q83. Find the average salary of each department using a join.

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary FROM departments d JOIN employees e ON d.dept_id = e.dept_id GROUP BY d.dept_name;
```

SAMPLE OUTPUT:

dept_name	avg_salary
Finance	55000.0
HR	55000.0
IT	67500.0

Q84. Find the highest-paid employee in each department.

```
SELECT d.dept_name, MAX(e.salary) AS max_sal FROM departments d JOIN employees e ON d.dept_id = e.dept_id GROUP BY d.dept_name;
```

SAMPLE OUTPUT:

dept_name	max_sal
Finance	75000
HR	60000
IT	120000

Q85. Find employees earning more than 60,000 along with their department names.

```
SELECT e.emp_name, e.salary, d.dept_name FROM employees e JOIN departments d ON
e.dept_id = d.dept_id WHERE e.salary > 60000;
```

SAMPLE OUTPUT:

emp_name	salary	dept_name
Sarah Connor	120000	IT
Diana	75000	Finance
Ian	150000	Sales

Q86. Find employees who earn more than their manager.

```
SELECT e.emp_name FROM employees e JOIN employees m ON e.manager_id = m.emp_id WHERE
e.salary > m.salary;
```

SAMPLE OUTPUT:

emp_name
Evan
Ian

Q87. Find employees who earn less than their manager.

```
SELECT e.emp_name FROM employees e JOIN employees m ON e.manager_id = m.emp_id WHERE
e.salary < m.salary;
```

SAMPLE OUTPUT:

emp_name
Alice Smith
Brian
Alan

Q88. Find employees who have the same salary as their manager.

```
SELECT e.emp_name FROM employees e JOIN employees m ON e.manager_id = m.emp_id WHERE
e.salary = m.salary;
```

SAMPLE OUTPUT:

OUTPUT: (0 ROWS RETURNED)

Q89. Find employees who report directly to a particular manager.

```
SELECT e.emp_name FROM employees e JOIN employees m ON e.manager_id = m.emp_id WHERE
m.emp_name = 'Sarah Connor';
```

SAMPLE OUTPUT:

emp_name
Alice Smith
Brian
Dan

Q90. Find managers who have more than five employees.

```
SELECT m.emp_name, COUNT(e.emp_id) AS direct_reports FROM employees e JOIN employees m
ON e.manager_id = m.emp_id GROUP BY m.emp_id, m.emp_name HAVING COUNT(e.emp_id) > 5;
```

SAMPLE OUTPUT:

emp_name	direct_reports
Sarah Connor	6

Q91. Find departments and their total salary expenditure.

```
SELECT d.dept_name, SUM(e.salary) AS total_exp FROM departments d LEFT JOIN employees e
ON d.dept_id = e.dept_id GROUP BY d.dept_id, d.dept_name;
```

SAMPLE OUTPUT:

dept_name	total_exp
IT	270000
HR	110000
Finance	110000

Q92. Find departments with no employees using a LEFT JOIN.

```
SELECT d.dept_name FROM departments d LEFT JOIN employees e ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

SAMPLE OUTPUT:

dept_name
Operations
EmptyDept

Q93. Find customers who have placed at least one order.

```
SELECT DISTINCT c.customer_name FROM customers c JOIN orders o ON c.customer_id =
o.customer_id;
```

SAMPLE OUTPUT:

customer_name
Whale Customer
Regular Joe

Q94. Find customers who have never placed an order.

```
SELECT c.customer_name FROM customers c LEFT JOIN orders o ON c.customer_id =
o.customer_id WHERE o.order_id IS NULL;
```

SAMPLE OUTPUT:

customer_name
Window Shopper

Q95. Find the total number of orders placed by each customer.

```
SELECT c.customer_name, COUNT(o.order_id) AS total_orders FROM customers c LEFT JOIN
orders o ON c.customer_id = o.customer_id GROUP BY c.customer_id, c.customer_name;
```

SAMPLE OUTPUT:

customer_name	total_orders
Whale Customer	6
Regular Joe	1
Window Shopper	0

Q96. Find customers who have placed more than five orders.

```
SELECT c.customer_name FROM customers c JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name HAVING COUNT(o.order_id) > 5;
```

SAMPLE OUTPUT:

customer_name
Whale Customer

Q97. Find the total amount spent by each customer.

```
SELECT c.customer_name, COALESCE(SUM(o.total_amount), 0) AS total_spent FROM customers
c LEFT JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.customer_id,
c.customer_name;
```

SAMPLE OUTPUT:

customer_name	total_spent
Whale Customer	6150
Regular Joe	500
Window Shopper	0

Q98. Find customers whose total spending exceeds 100,000.

```
SELECT c.customer_name FROM customers c JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name HAVING SUM(o.total_amount) > 100000;
```

SAMPLE OUTPUT:

OUTPUT: (0 ROWS RETURNED)

Q99. Find products that have never been ordered.

```
SELECT p.product_name FROM products p LEFT JOIN order_items oi ON p.product_id =
oi.product_id WHERE oi.order_id IS NULL;
```

SAMPLE OUTPUT:

product_name
T-Shirt
Unsold Item

Q100. Find the most frequently ordered product.

```
SELECT p.product_name FROM products p JOIN order_items oi ON p.product_id =
oi.product_id GROUP BY p.product_id, p.product_name ORDER BY SUM(oi.quantity) DESC
LIMIT 1;
```

SAMPLE OUTPUT:

product_name
Laptop

Q101. Find the total quantity sold for each product.

```
SELECT p.product_name, COALESCE(SUM(oi.quantity), 0) AS units_sold FROM products p LEFT JOIN order_items oi ON p.product_id = oi.product_id GROUP BY p.product_id, p.product_name;
```

SAMPLE OUTPUT:

product_name	units_sold
Laptop	5
Phone	3
SQL Book	3

Q102. Find the total revenue generated by each product.

```
SELECT p.product_name, COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS revenue FROM products p LEFT JOIN order_items oi ON p.product_id = oi.product_id GROUP BY p.product_id, p.product_name;
```

SAMPLE OUTPUT:

product_name	revenue
Laptop	5000
Phone	1500
SQL Book	150

Q103. Find the top five products by revenue.

```
SELECT p.product_name FROM products p JOIN order_items oi ON p.product_id = oi.product_id GROUP BY p.product_id, p.product_name ORDER BY SUM(oi.quantity * oi.unit_price) DESC LIMIT 5;
```

SAMPLE OUTPUT:

product_name
Laptop
Phone
SQL Book

Q104. Find employees assigned to more than three projects.

```
SELECT e.emp_name FROM employees e JOIN employee_projects ep ON e.emp_id = ep.emp_id
GROUP BY e.emp_id, e.emp_name HAVING COUNT(ep.project_id) > 3;
```

SAMPLE OUTPUT:

emp_name
Sarah Connor
Diana

Q105. Find projects with more than five employees assigned.

```
SELECT p.project_name FROM projects p JOIN employee_projects ep ON p.project_id =
ep.project_id GROUP BY p.project_id, p.project_name HAVING COUNT(ep.emp_id) > 5;
```

SAMPLE OUTPUT:

OUTPUT: (0 ROWS RETURNED)

Q106. Find employees who are not assigned to any project.

```
SELECT e.emp_name FROM employees e LEFT JOIN employee_projects ep ON e.emp_id =
ep.emp_id WHERE ep.project_id IS NULL;
```

SAMPLE OUTPUT:

emp_name
Orphan Employee

Q107. Find projects that currently have no employees.

```
SELECT p.project_name FROM projects p LEFT JOIN employee_projects ep ON p.project_id =
ep.project_id WHERE ep.emp_id IS NULL;
```

SAMPLE OUTPUT:

project_name
Unstaffed Project

Q108. Find employees working on projects managed by their own department.

```
SELECT e.emp_name FROM employees e JOIN employee_projects ep ON e.emp_id = ep.emp_id
JOIN projects p ON ep.project_id = p.project_id WHERE e.dept_id = p.dept_id;
```

SAMPLE OUTPUT:

emp_name
Sarah Connor
Sarah Connor
Alice Smith

Q109. Find the department whose employees have generated the highest project revenue.

```
SELECT d.dept_name FROM departments d JOIN employees e ON d.dept_id = e.dept_id JOIN
employee_projects ep ON e.emp_id = ep.emp_id GROUP BY d.dept_id, d.dept_name ORDER BY
SUM(ep.revenue_generated) DESC LIMIT 1;
```

SAMPLE OUTPUT:

dept_name
IT

Q110. Find the customer who has spent the most money on products.

```
SELECT c.customer_name FROM customers c JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name ORDER BY SUM(o.total_amount) DESC LIMIT 1;
```

SAMPLE OUTPUT:

customer_name
Whale Customer